Go Programming Language

UNIT I

- Prepared By
- Mrs. Rutuja Chavan
- Dept. of BCA(Sci)

UNIT I

Full GO Course

https://youtu.be/yyUHQIec83I

1. Go Runtime and Compilations
2. Keywords and Identifiers
3. Constants and Variables
4. Operators and Expressions
5. Local Assignments
6. Booleans, Numeric , Characters
7. Pointers and Addresses Video link : https://youtu.be/sTFJtxJXkaY
8. Strings
9. if-else, switch, for loop
10. Iterations
11. Using break and continue

Go Get Started

- To start using Go, you need two things:
- A text editor, like VS Code, to write Go code
- A compiler, like GCC, to translate the Go code into a language that the computer will understand
- There are many text editors and compilers to choose from. In this tutorial, we will use an IDE (see below).

Go Install

- You can find the relevant installation files at https://golang.org/dl/.

- Follow the instructions related to your operating system. To check if Go was installed successfully, you can run the following command in a terminal window:

```
go version
```

Which should show the version of your Go installation.

Go Install IDE

- An IDE (Integrated Development Environment) is used to edit AND compile the code.
- Popular IDE's include Visual Studio Code (VS Code), Vim, Eclipse, and Notepad. These are all free, and they can be used to both edit and debug Go code.
- **Note:** Web-based IDE's can work as well, but functionality is limited.
- We will use **VS Code** in our tutorial, which we believe is a good place to start.
- You can find the latest version of VS Code at https://code.visualstudio.com/.

Go Quickstart

Let's create our first Go program.

- Launch the VS Code editor
- Open the extension manager or alternatively, press `Ctrl + Shift + x`
- In the search box, type "go" and hit enter
- Find the Go extension by the GO team at Google and install the extension
- After the installation is complete, open the command palette by pressing `Ctrl + Shift + p`
- Run the `Go: Install/Update Tools` command
- Select all the provided tools and click OK

VS Code is now configured to use Go.

Open up a terminal window and type:

```
go mod init example.com/hello
```

Do not worry if you do not understand why we type the above command. Just think of it as something that you always do, and that you will learn more about in a later chapter.

Create a new file (`File > New File`). Copy and paste the following code and save the file as `helloworld.go` (`File > Save As`):

helloworld.go

```
package main
import ("fmt")

func main() {
  fmt.Println("Hello World!")
}
```

Now, run the code: Open a terminal in VS Code and type:

```
go run .\helloworld.go
    Hello World!
```

- **Congratulations**! You have now written and executed your first Go program.
- If you want to save the program as an executable, type and run:
- `go build .\helloworld.go`

Go Programming Language

- Golang or Go Programming Language is a statically-typed and procedural programming language having syntax similar to **C** language.
- It was developed in 2007 by Robert Griesemer, Rob Pike, and Ken Thompson at Google. But they launched it in 2009 as an open-source programming language.
- It provides a rich standard library, garbage collection, and dynamic-typing capability , many advanced built-in types such as variable length arrays and key-value maps etc. and also provides support for the environment adopting patterns alike to dynamic languages.
- The latest version of the Golang is **1.13.1** released on 3rd September 2019. Here, we are providing a complete tutorial of Golang with proper examples.

GO SYNTAX

- Go is modern, fast and comes with a powerful standard library.
- Go has built-in concurrency.
- Go uses interfaces as the building blocks of code reusability.

The basic structure of a Go programs consists of following parts:-

- Package Declaration
- Import Packages
- Variables
- Statements and Expressions
- Functions
- Comments

Go Example

Let's see a simple example of Go programming language.

```
package main
import "fmt"
func main() {
   fmt.Println("Hello, World")
}
```

Output:

```
Hello, World
```

Example explained

Line 1: In Go, every program is part of a package. We define this using the `package` keyword. In this example, the program belongs to the `main` package.

Line 2: `import ("fmt")` lets us import files included in the `fmt` package.

Line 3: A blank line. Go ignores white space. Having white spaces in code makes it more readable.

Line 4: `func main() {}` is a function. Any code inside its curly brackets `{}` will be executed.

Line 5: `fmt.Println()` is a function made available from the `fmt` package. It is used to output/print text. In our example it will output "Hello World!".

Note: In Go, any executable code belongs to the `main` package.

Go Statements

`fmt.Println("Hello World!")` is a statement.
In Go, statements are separated by ending a line (hitting the Enter key) or by a semicolon ";".

Hitting the Enter key adds ";" to the end of the line implicitly (does not show up in the source code).

The left curly bracket { cannot come at the start of a line.
Run the following code and see what happens:

- Example

```
package main
import ("fmt")

func main()
{
  fmt.Println("Hello World!")
}
```

Go Compact Code

- You can write more compact code, like shown below (this is not recommended because it makes the code more difficult to read):

- Example

- `package main; import ("fmt"); func main() { fmt.Println("Hello World!");}`

Go Comments

- A comment is a text that is ignored upon execution.
- Comments can be used to explain the code, and to make it more readable.
- Comments can also be used to prevent code execution when testing an alternative code.
- Go supports single-line or multi-line comments.

Go Single-line Comments

Single-line comments start with two forward slashes (//).
Any text between // and the end of the line is ignored by the compiler (will not be executed).

Example

```
// This is a comment
package main
import ("fmt")

func main() {
  // This is a comment
  fmt.Println("Hello World!")
}
```

Go Multi-line Comments

Multi-line comments start with /* and ends with */.
Any text between /* and */ will be ignored by the compiler:

Example

```
package main
import ("fmt")

func main() {
  /* The code below will print Hello World
  to the screen, and it is amazing */
  fmt.Println("Hello World!")
}
```

Tip: It is up to you which you want to use. Normally, we use // for short comments, and /* */ for longer comments.

Comment to Prevent Code Execution

- You can also use comments to prevent the code from being executed.
- The commented code can be saved for later reference and troubleshooting.
- Example

```
package main
import ("fmt")

func main() {
  fmt.Println("Hello World!")
  // fmt.Println("This line does not execute")
}
```

Why Golang?

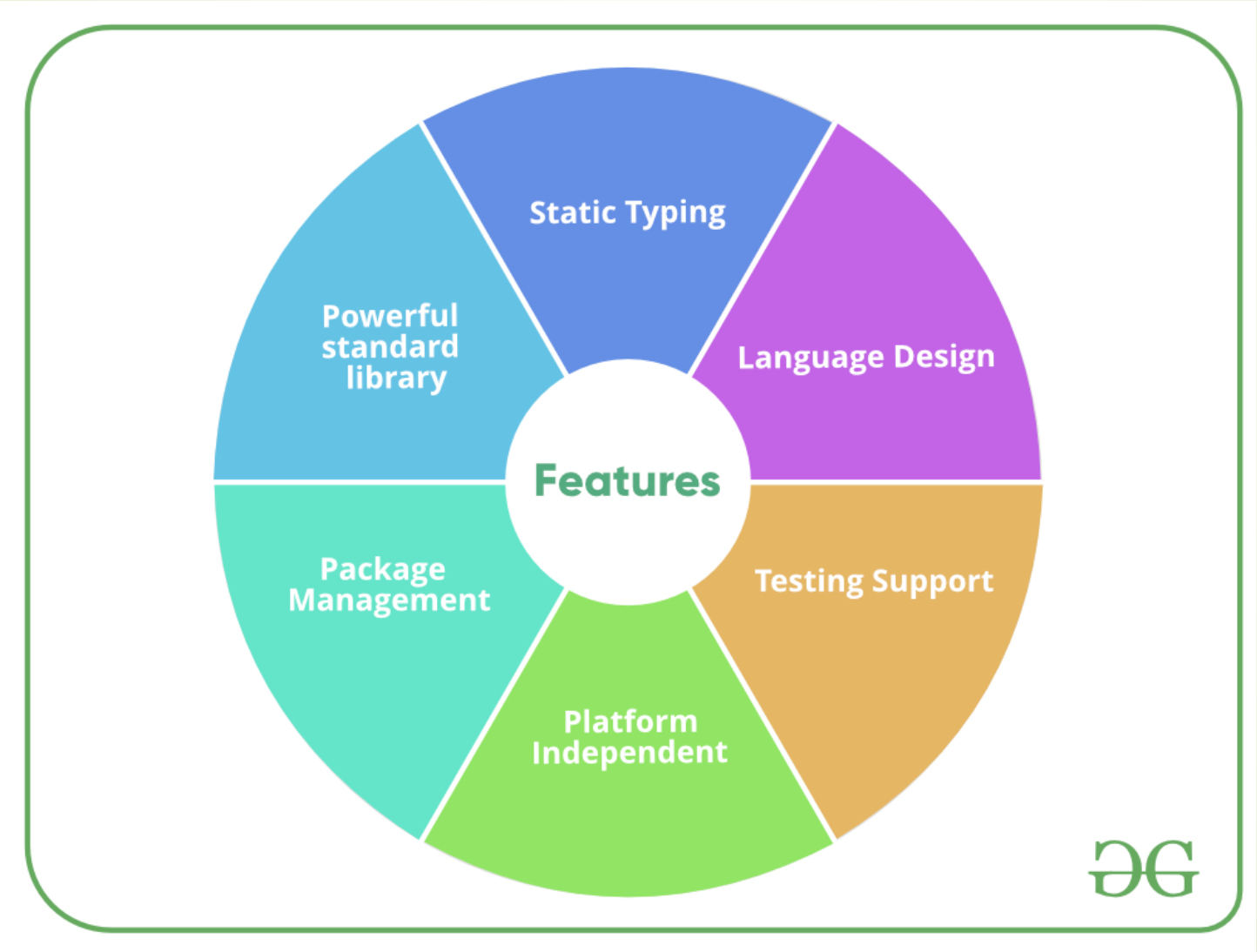
The main purpose of designing Golang was to eliminate the problems of existing languages. So let us see the problems that we are facing with Python, Java, C/C++ programming languages:

- **Python:** It is easy to use but slow in compare to Golang.
- **Java:** It has very complex type system.
- **C/C++:** It has slow compilation time as well as complex type system.

- Also, all these languages were designed when multi-threading applications were rare, so not much effective to highly scalable, concurrent and parallel applications.

- Threading consumes 1MB whereas Goroutine consumes 2KB of memory, hence at the same time, we can have millions of goroutine triggered.

Key Features



- After completing the installation process, any IDE or text editor can be used to write Golang Codes and Run them on the IDE or the Command prompt with the use of command:

```
go run filename.go
```

Executing Hello World! Program

- To run a Go program you need a Go compiler.
- In Go compiler, first you create a program and save your program with extension **.go**

  for example, **first.go**.

```
// First Go program
package main

import "fmt"

// Main function
func main() {

    fmt.Println("!... Hello World ...!")
}
```

Run this **first.go** file in the go compiler using the following command, i.e:

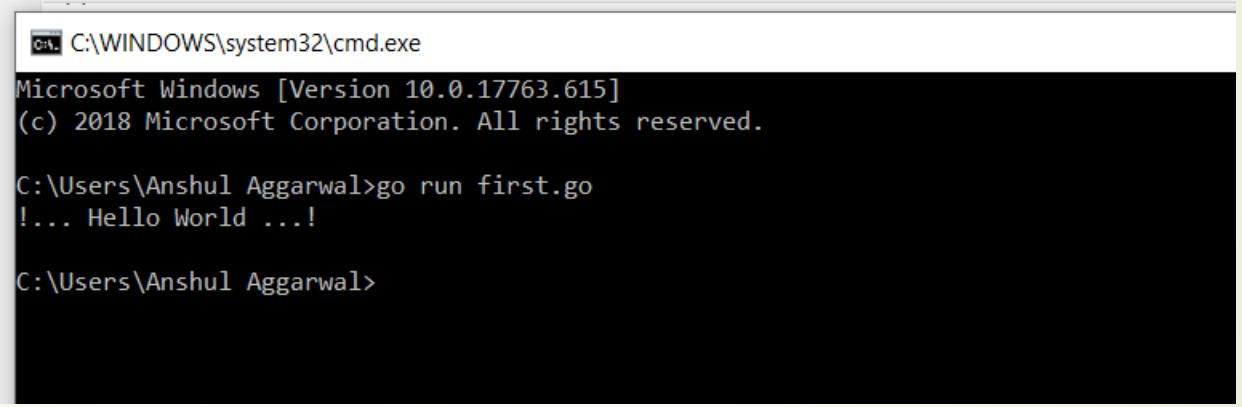
```
$ go run first.go
```

- **Output:**

```
!... Hello World ...!
```

- The First line is the package deceleration, here the name of the package is main. Package deceleration is mandatory for all the go programs.
- The next line is an import statement, here we are importing "fmt". The compiler will include the files of the package fmt.
- The next line is a main() function, all execution begins with the main function.
- The next line fmt.Println(...) is a function available in Go. This function will print the message "Hello, World" on the screen.

Output screen will look like this……



Go Variable Types

In Go, there are different **types** of variables, for example:

- `int` - stores integers (whole numbers), such as 123 or -123
- `float32` - stores floating point numbers, with decimals, such as 19.99 or -19.99
- `string` - stores text, such as "Hello World". String values are surrounded by double quotes
- `bool` - stores values with two states: true or false

Declaring (Creating) Variables

In Go, there are two ways to declare a variable:

1. With the `var` keyword:

Use the `var` keyword, followed by variable name and type:

Syntax

```
var variablename type = value
```

Note: You always have to specify either `type` or `value` (or both).

2. With the `:=` sign:

Use the `:=` sign, followed by the variable value:

Syntax

```
variablename := value
```

Note: In this case, the type of the variable is **inferred** from the value (means that the compiler decides the type of the variable, based on the value).

Note: It is not possible to declare a variable using `:=`, without assigning a value to it.

Variable Declaration With Initial Value

- If the value of a variable is known from the start, you can declare the variable and assign a value to it on one line:
- Example
- ```
  package main
  import ("fmt")

  func main() {
    var student1 string = "John" //type is string
    var student2 = "Jane" //type is inferred
    x := 2 //type is inferred

    fmt.Println(student1)
    fmt.Println(student2)
    fmt.Println(x)
  }
  ```

Note: The variable types of `student2` and `x` is **inferred** from their values.
  ```

Variable Declaration Without Initial Value

- In Go, all variables are initialized.
- So, if you declare a variable without an initial value, its value will be set to the default value of its type:

Example:

```
package main
import ("fmt")

func main() {
  var a string
  var b int
  var c bool

  fmt.Println(a)
  fmt.Println(b)
  fmt.Println(c)
}
```

Example explained

In this example there are 3 variables:

- a
- b
- c

These variables are declared but they have not been assigned initial values.

By running the code, we can see that they already have the default values of their respective types:

- a is ""
- b is 0
- c is false

Value Assignment After Declaration

- It is possible to assign a value to a variable after it is declared. This is helpful for cases the value is not initially known.

Example:

```
package main
import ("fmt")

func main() {
  var student1 string
  student1 = "John"
  fmt.Println(student1)
}
```

Note: It is not possible to declare a variable using ":=" without assigning a value to it.

Difference Between var and :=

- There are some small differences between the `var` var `:=`:

var	:=
Can be used **inside** and **outside** of functions	Can only be used **inside** functions
Variable declaration and value assignment **can be done separately**	Variable declaration and value assignment **cannot be done separately** (must be done in the same line)

Example

- This example shows declaring variables outside of a function, with the `var` keyword:

Example:

```
package main
import ("fmt")

var a int
var b int = 2
var c = 3

func main() {
  a = 1
  fmt.Println(a)
  fmt.Println(b)
  fmt.Println(c)
}
```

Example

Since `:=` is used outside of a function, running the program results in an error.

```
package main
import ("fmt")
a := 1
func main() {
  fmt.Println(a)
}
```

Result:

```
./prog.go:5:1: syntax error:
non-declaration statement
outside function body
```

Go Multiple Variable Declaration

In Go, it is possible to declare multiple variables in the same line.

Example

This example shows how to declare multiple variables in the same line:

```
package main
import ("fmt")

func main() {
  var a, b, c, d int = 1, 3, 5, 7

  fmt.Println(a)
  fmt.Println(b)
  fmt.Println(c)
  fmt.Println(d)
}
```

Note: If you use the `type` keyword, it is only possible to declare **one type** of variable per line.

If the type keyword is not specified, you can declare different types of variables in the same line:

Example

```
package main
import ("fmt")

func main() {
  var a, b = 6, "Hello"
  c, d := 7, "World!"

  fmt.Println(a)
  fmt.Println(b)
  fmt.Println(c)
  fmt.Println(d)
}
```

Go Variable Declaration in a Block

- Multiple variable declarations can also be grouped together into a block for greater readability:
  Example

```
package main
import ("fmt")

func main() {
    var (
      a int
      b int = 1
      c string = "hello"
    )

  fmt.Println(a)
  fmt.Println(b)
  fmt.Println(c)
}
```

Go Variable Naming Rules

A variable can have a short name (like x and y) or a more descriptive name (age, price, carname, etc.).

Go variable naming rules:

1. A variable name must start with a letter or an underscore character (_)
2. A variable name cannot start with a digit
3. A variable name can only contain alpha-numeric characters and underscores (`a-z, A-Z`, `0-9`, and `_`)
4. Variable names are case-sensitive (age, Age and AGE are three different variables)
5. There is no limit on the length of the variable name
6. A variable name cannot contain spaces
7. The variable name cannot be any Go keywords

Multi-Word Variable Names

- Variable names with more than one word can be difficult to read.
- There are several techniques you can use to make them more readable:

1. Camel Case

Each word, except the first, starts with a capital letter:

```
myVariableName = "John“
```

2. Pascal Case

Each word starts with a capital letter:

```
MyVariableName = "John“
```

3. Snake Case

Each `my_variable_name` word is separated by an underscore character:

```
= "John"
```

Go Constants

If a variable should have a fixed value that cannot be changed, you can use the `const` keyword.

The `const` keyword declares the variable as "constant", which means that it is **unchangeable and read-only**.

Syntax

```
const CONSTNAME type = value
```

- **Note:** The value of a constant must be assigned when you declare it.

- Declaring a Constant
- Here is an example of declaring a constant in Go:
- Example
- ```
  package main
  import ("fmt")

  const PI = 3.14

  func main() {
    fmt.Println(PI)
  }
  ```
  ```

Constant Rules

- Constant names follow the same naming rules as variables
- Constant names are usually written in uppercase letters (for easy identification and differentiation from variables)
- Constants can be declared both inside and outside of a function

Constant Types

There are two types of constants:

- Typed constants
- Untyped constants

Typed Constants

Typed constants are declared with a defined type:

Example

```
package main
import ("fmt")

const A int = 1

func main() {
  fmt.Println(A)
}
```

Untyped Constants

Untyped constants are declared without a type:

Example

```
package main
import ("fmt")

const A = 1

func main() {
  fmt.Println(A)
}
```

Note: In this case, the type of the constant is inferred from the value (means the compiler decides the type of the constant, based on the value).

Constants: Unchangeable and Read-only

When a constant is declared, it is not possible to change the value later:

Example

```
package main
import ("fmt")

func main() {
  const A = 1
  A = 2
  fmt.Println(A)
}
```

Result:

```
./prog.go:8:7: cannot assign to A
```

Multiple Constants Declaration

Multiple constants can be grouped together into a block for readability:
Example

```
package main
import ("fmt")

const (
  A int = 1
  B = 3.14
  C = "Hi!"
)

func main() {
  fmt.Println(A)
  fmt.Println(B)
  fmt.Println(C)
}
```

Go Output Functions

Go has three functions to output text:

- `Print()`
- `Println()`
- `Printf()`

The Print() Function

The `Print()` function prints its arguments with their default format.

Example

Print the values of `i` and `j`:

```
package main
import ("fmt")

func main() {
  var i,j string = "Hello","World"

  fmt.Print(i)
  fmt.Print(j)
}
```

Result:

```
HelloWorld
```

Example

If we want to print the arguments in new lines, we need to use \n.

```
package main
import ("fmt")

func main() {
  var i,j string = "Hello","World"

  fmt.Print(i, "\n")
  fmt.Print(j, "\n")
}
```

Tip: \n creates new lines.

Output:

```
Hello
World
```

Example

It is also possible to only use one `Print()` for printing multiple variables.

```
package main
import ("fmt")

func main() {
  var i,j string = "Hello","World"

  fmt.Print(i, "\n",j)
}
```

Result:

```
Hello
World
```

Example

If we want to add a space between string arguments, we need to use " ":

```
package main
import ("fmt")

func main() {
  var i,j string = "Hello","World"

  fmt.Print(i, " ", j)
}
```

Result:

```
Hello World
```

Example

Print() inserts a space between the arguments if **neither** are strings:

```
package main
import ("fmt")

func main() {
  var i,j = 10,20

  fmt.Print(i,j)
}
```

Result:

```
10 20
```

The Println() Function

The `Println()` function is similar to `Print()` with the difference that a whitespace is added between the arguments, and a newline is added at the end:

Example

```
package main
import ("fmt")

func main() {
  var i,j string = "Hello","World"

  fmt.Println(i,j)
}
```

Result:

```
Hello World
```

The Printf() Function

The `Printf()` function first formats its argument based on the given formatting verb and then prints them.
Here we will use two formatting verbs:

- `%v` is used to print the **value** of the arguments
- `%T` is used to print the **type** of the arguments

Example

```
package main
import ("fmt")

func main() {
  var i string = "Hello"
  var j int = 15

  fmt.Printf("i has value: %v and type: %T\n", i, i)
  fmt.Printf("j has value: %v and type: %T", j, j)
}
```

Result:

```
i has value: Hello and type: string
j has value: 15 and type: int
```

Formatting Verbs for Printf()

Go offers several formatting verbs that can be used with the `Printf()` function.

General Formatting Verbs

The following verbs can be used with all data types:

Verb	Description
%v	Prints the value in the default format
%#v	Prints the value in Go-syntax format
%T	Prints the type of the value
%%	Prints the % sign

Example

```
package main
import ("fmt")

func main() {
  var i = 15.5
  var txt = "Hello World!"

  fmt.Printf("%v\n", i)
  fmt.Printf("%#v\n", i)
  fmt.Printf("%v%%\n", i)
  fmt.Printf("%T\n", i)

  fmt.Printf("%v\n", txt)
  fmt.Printf("%#v\n", txt)
  fmt.Printf("%T\n", txt)
}
```

```
15.5
15.5
15.5%
float64
Hello World!
"Hello World!"
string
```

Integer Formatting Verbs

- The following verbs can be used with the integer data type:

Verb	Description
%b	Base 2
%d	Base 10
%+d	Base 10 and always show sign
%o	Base 8
%O	Base 8, with leading 0o
%x	Base 16, lowercase
%X	Base 16, uppercase
%#x	Base 16, with leading 0x
%4d	Pad with spaces (width 4, right justified)
%-4d	Pad with spaces (width 4, left justified)
%04d	Pad with zeroes (width 4

Integer Formatting Verbs : Example

```
package main
import ("fmt")

func main() {
  var i = 15

  fmt.Printf("%b\n", i)
  fmt.Printf("%d\n", i)
  fmt.Printf("%+d\n", i)
  fmt.Printf("%o\n", i)
  fmt.Printf("%O\n", i)
  fmt.Printf("%x\n", i)
  fmt.Printf("%X\n", i)
  fmt.Printf("%#x\n", i)
  fmt.Printf("%4d\n", i)
  fmt.Printf("%-4d\n", i)
  fmt.Printf("%04d\n", i)
}
```

Result:

```
1111
15
+15
17
0o17
f
F
0xf
  15
15
0015
```

String Formatting Verbs

- The following verbs can be used with the string data type:

Verb	Description
%s	Prints the value as plain string
%q	Prints the value as a double-quoted string
%8s	Prints the value as plain string (width 8, right justified)
%-8s	Prints the value as plain string (width 8, left justified)
%x	Prints the value as hex dump of byte values
% x	Prints the value as hex dump with spaces

String Formatting Verbs : Example

```
package main
import ("fmt")

func main() {
  var txt = "Hello"

  fmt.Printf("%s\n", txt)
  fmt.Printf("%q\n", txt)
  fmt.Printf("%8s\n", txt)
  fmt.Printf("%-8s\n", txt)
  fmt.Printf("%x\n", txt)
  fmt.Printf("% x\n", txt)
}
```

Result:

```
Hello
"Hello"
   Hello
Hello
48656c6c6f
48 65 6c 6c 6f
```

Boolean Formatting Verbs:

- The following verb can be used with the boolean data type:

Verb	Description
%t	Value of the boolean operator in true or false format (same as using %v)

Boolean Formatting Verbs: Example

```
package main
import ("fmt")

func main() {
  var i = true
  var j = false

  fmt.Printf("%t\n", i)
  fmt.Printf("%t\n", j)
}
```

Result:

```
true
false
```

Float Formatting Verbs

- The following verbs can be used with the float data type:

Verb	Description
%e	Scientific notation with 'e' as exponent
%f	Decimal point, no exponent
%.2f	Default width, precision 2
%6.2f	Width 6, precision 2
%g	Exponent as needed, only necessary digits

Float Formatting Verbs: Example

```go
package main
import ("fmt")

func main() {
  var i = 3.141

  fmt.Printf("%e\n", i)
  fmt.Printf("%f\n", i)
  fmt.Printf("%.2f\n", i)
  fmt.Printf("%6.2f\n", i)
  fmt.Printf("%g\n", i)
}
```

Result:

```
3.141000e+00
3.141000
3.14
  3.14
3.141
```

Data Types

- Data types specify the type of data that a valid Go variable can hold. In Go language, the type is divided into four categories which are as follows:

1. **Basic type:** Numbers, strings, and booleans come under this category.
2. **Aggregate type:** Array and structs come under this category.
3. **Reference type:** Pointers, slices, maps, functions, and channels come under this category.
4. **Interface type**

Here, we will discuss *Basic Data Types* in the Go language. The ***Basic Data Types*** are further categorized into three subcategories which are:

1. **Numbers**
2. **Booleans**
3. **Strings**

Go Data Types

- Data type specifies the size and type of variable values.

- Go is statically typed, meaning that once a variable type is defined, it can only store data of that type.

- Go has three basic data types:

1. **bool**: represents a boolean value and is either true or false
2. **Numeric**: represents integer types, floating point values, and complex types
3. **string**: represents a string value

Data Types: Example

This example shows some of the different data types in Go:

```
package main
import ("fmt")

func main() {
  var a bool = true      // Boolean
  var b int = 5          // Integer
  var c float32 = 3.14   // Floating point number
  var d string = "Hi!"   // String

  fmt.Println("Boolean: ", a)
  fmt.Println("Integer: ", b)
  fmt.Println("Float:   ", c)
  fmt.Println("String:  ", d)
}
```

Result:

```
Boolean: true

Integer: 5

Float: 3.14

String: Hi!
```

Boolean Data Type

- A boolean data type is declared with the `bool` keyword and can only take the values `true` or `false`.
- The default value of a boolean data type is `false`.
- This example shows some different ways to declare Boolean variables:

Example

```
package main
import ("fmt")

func main() {
  var b1 bool = true // typed declaration with initial value
  var b2 = true // untyped declaration with initial value
  var b3 bool // typed declaration without initial value
  b4 := true // untyped declaration with initial value

  fmt.Println(b1) // Returns true
  fmt.Println(b2) // Returns true
  fmt.Println(b3) // Returns false
  fmt.Println(b4) // Returns true
}
```

Example

```
package main
import ("fmt")

func main() {
  var b1 bool = true // typed declaration with initial value
  var b2 = true // untyped declaration with initial value
  var b3 bool // typed declaration without initial value
  b4 := true // untyped declaration with initial value

  fmt.Println(b1) // Returns true
  fmt.Println(b2) // Returns true
  fmt.Println(b3) // Returns false
  fmt.Println(b4) // Returns true
}
```

Result:

```
true
true
false
true
```

Go Integer Data Types

- Integer data types are used to store a whole number without decimals, like 35, -50, or 1345000.

- The integer data type has two categories:

1. **Signed integers** - can store both positive and negative values
2. **Unsigned integers** - can only store non-negative values

Tip: The default type for integer is `int`. If you do not specify a type, the type will be `int`.

Signed Integers

- Signed integers, declared with one of the `int` keywords, can store both positive and negative values:

Example

```
package main
import ("fmt")

func main() {
  var x int = 500
  var y int = -4500
  fmt.Printf("Type: %T, value: %v", x, x)
  fmt.Printf("Type: %T, value: %v", y, y)
}
```

Result:

```
Type: int, value: 500
Type: int, value: -4500
```

Signed Integers

Go has five keywords/types of signed integers:

Type	Size	Range
int	Depends on platform: 32 bits in 32 bit systems and 64 bit in 64 bit systems	-2147483648 to 2147483647 in 32 bit systems and -9223372036854775808 to 9223372036854775807 in 64 bit systems
int8	8 bits/1 byte	-128 to 127
int16	16 bits/2 byte	-32768 to 32767
int32	32 bits/4 byte	-2147483648 to 2147483647
int64	64 bits/8 byte	-9223372036854775808 to 9223372036854775807

Unsigned Integers

Unsigned integers, declared with one of the `uint` keywords, can only store non-negative values:

Example

```
package main
import ("fmt")

func main() {
  var x uint = 500
  var y uint = 4500
  fmt.Printf("Type: %T, value: %v", x, x)
  fmt.Printf("Type: %T, value: %v", y, y)
}
```

- Result:

```
Type: uint, value: 500
Type: uint, value: 4500
```

Unsigned Integers

- Go has five keywords/types of unsigned integers:

Type	Size	Range
uint	Depends on platform: 32 bits in 32 bit systems and 64 bit in 64 bit systems	0 to 4294967295 in 32 bit systems and 0 to 18446744073709551615 in 64 bit systems
uint8	8 bits/1 byte	0 to 255
uint16	16 bits/2 byte	0 to 65535
uint32	32 bits/4 byte	0 to 4294967295
uint64	64 bits/8 byte	0 to 18446744073709551615

Which Integer Type to Use?

- The type of integer to choose, depends on the value the variable has to store.

Example

- This example will result in an error because 1000 is out of range for int8 (which is from -128 to 127):

```
package main
import ("fmt")

func main() {
  var x int8 = 1000
  fmt.Printf("Type: %T, value: %v", x, x)
}
```

Result:

```
./prog.go:5:7: constant 1000 overflows int8
```

Go Float Data Types

- The float data types are used to store positive and negative numbers with a decimal point, like 35.3, -2.34, or 3597.34987.
- The float data type has two keywords:

Type	Size	Range
float32	32 bits	-3.4e+38 to 3.4e+38.
float64	64 bits	-1.7e+308 to +1.7e+308.

- **Tip:** The default type for float is `float64`. If you do not specify a type, the type will be `float64`.

The float32 Keyword

Example

This example shows how to declare some variables of type `float32`:

```
package main
import ("fmt")

func main() {
  var x float32 = 123.78
  var y float32 = 3.4e+38
  fmt.Printf("Type: %T, value: %v\n", x, x)
  fmt.Printf("Type: %T, value: %v", y, y)
}
```

Result:

```
Type: float32, value: 123.78
Type: float32, value: 3.4e+38
```

The float64 Keyword

The `float64` data type can store a larger set of numbers than `float32`.

Example

This example shows how to declare a variable of type `float64`:

```
package main
import ("fmt")

func main() {
  var x float64 = 1.7e+308
  fmt.Printf("Type: %T, value: %v", x, x)
}
```

Result:

```
Type: float64, value: 1.7e+308
```

Which Float Type to Use?

- The type of float to choose, depends on the value the variable has to store.
- Example

This example will result in an error because 3.4e+39 is out of range for float32:

```
package main
import ("fmt")

func main() {
  var x float32= 3.4e+39
  fmt.Println(x)
}
```

Result:

```
./prog.go:5:7: constant 3.4e+39 overflows
float32
```

String Data Type

- The `string` data type is used to store a sequence of characters (text). String values must be surrounded by double quotes:
- Example

```
package main
import ("fmt")

func main() {
  var txt1 string = "Hello!"
  var txt2 string
  txt3 := "World 1"

  fmt.Printf("Type: %T, value: %\n", txt1, txt1)
  fmt.Printf("Type: %T, value: %v\n", txt2, txt2)
  fmt.Printf("Type: %T, value: %v\n", txt3, txt3)
}
```

Result:

```
Type: string, value: Hello!
Type: string, value:
Type: string, value: World 1
```

- **Complex Numbers:**
- The complex numbers are divided into two parts are shown in the below table.

float32 and float64 are also part of these complex numbers.

The in-built function creates a complex number from its imaginary and real part and in-built imaginary and real function extract those parts.

DATA TYPE	DESCRIPTION
complex64	Complex numbers which contain float32 as a real and imaginary component.
complex128	Complex numbers which contain float64 as a real and imaginary component.

Identifiers and Keywords

- Identifiers are the user-defined name of the program components.
- In Go language, an identifier can be a variable name, function name, constant, statement labels, package name, or types.

Example:

```
// Valid identifiers:
_geeks23
geeks
gek23sd
Geeks
geeKs
geeks_geeks

// Invalid identifiers:

212geeks
if
default
```

- Keywords or Reserved words are the words in a language that are used for some internal process or represent some predefined actions. These words are therefore not allowed to use as an identifier.

  Doing this will result in a compile-time error.

- There are ***total 25 keywords present*** in the Go language as follows:

break	default	func	interface	select
case	defer	go	map	struct
chan	else	goto	package	switch
const	fallthrough	if	range	type
continue	for	import	return	var

Rules for Naming Variables:

- A Variable is a placeholder of the information which can be changed at runtime. And variables allow to Retrieve and Manipulate the stored information.
- **Rules for Naming Variables:**
- Variable names must begin with a letter or an underscore(_). And the names may contain the letters ‘a-z’ or ’A-Z’ or digits 0-9 as well as the character ‘_’.

```
Geeks, geeks, _geeks23  // valid variable
123Geeks, 23geeks         // invalid
variable
```

- A variable name should not start with a digit.

```
234geeks // illegal variable
```

- The name of the variable is case sensitive.

```
geeks and Geeks are two different variables
```

- Keywords is not allowed to use as a variable name.
- There is no limit on the length of the name of the variable, but it is advisable to use an optimum length of 4 – 15 letters only.

- There are two ways to declare a variable in Golang as follows:
- **1. Using var Keyword:** In Go language, variables are created using *var* keyword of a particular type, connected with name and provide its initial value.

Syntax:

```
var variable_name type = expression
// Go program to illustrate
// the use of var keyword
package main

import "fmt"

func main() {

// Variable declared and initialized without the  explicit type
var myvariable1 = 20

// Display the value and the type of the variables
fmt.Printf("The value of myvariable1 is : %d\n",myvariable1)

fmt.Printf("The type of myvariable1 is : %T\n", myvariable1)

}
```

Output:

```
The value of myvariable1 is : 20
The type of myvariable1 is : int
```

2. Using short variable declaration: The local variables which are declared and initialize in the functions are declared by using short variable declaration.

Syntax:

```
variable_name:= expression
```

- **Note:** Please don't confuse in between := and = as := is a declaration and = is assignment.

Example:

```
// Go program to illustrate the short variable
declaration
package main
import "fmt"

func main() {

// Using short variable declaration
myvar1 := 39

// Display the value and type of the variable
fmt.Printf("The value of myvar1 is : %d\n", myvar1)
fmt.Printf("The type of myvar1 is : %T\n", myvar1)

}
```

Output:

```
The value of myvar1 is : 39
The type of myvar1 is : int
```

Go Operators

- Operators are used to perform operations on variables and values. The + **operator** adds together two values, like in the example below:

- Example

```
package main
import ("fmt")

func main() {
  var a = 15 + 25
  fmt.Println(a)
}
```

O/p

40

- Although the + operator is often used to add together two values, it can also be used to add together a variable and a value, or a variable and another variable:

Example

```
package main
import ("fmt")

func main() {
  var (
    sum1 = 100 + 50    // 150 (100 + 50)
    sum2 = sum1 + 250  // 400 (150 + 250)
    sum3 = sum2 + sum2 // 800 (400 + 400)
  )
  fmt.Println(sum3)
}
```

O/P

```
800
```

Go divides the operators into the following groups:

- Arithmetic operators
- Assignment operators
- Comparison operators
- Logical operators
- Bitwise operators
- Misc Operators

These are used to perform common mathematical operations.

Arithmetic Operators

Operator	Name	Description	Example	Try it
+	Addition	Adds together two values	x + y	Try it »
-	Subtraction	Subtracts one value from another	x - y	Try it »
*	Multiplication	Multiplies two values	x * y	Try it »
/	Division	Divides one value by another	x / y	Try it »
%	Modulus	Returns the division remainder	x % y	Try it »
++	Increment	Increases the value of a variable by 1	x++	Try it »
--	Decrement	Decreases the value of a variable by 1	x--	Try it »

Example : ADDITION

```
package main
import ("fmt")
func main() {

 x:= 10
 y:= 2

 fmt.Println(x+y)
}
```

Example : SUBTRACTION

```
package main
import ("fmt")
func main() {

  x:= 10
  y:= 2

  fmt.Println(x-y)
}
```

Example : MULTIPLICATION

```
package main
import ("fmt")
func main() {

  x:= 10
  y:= 2

  fmt.Println(x*y)
}
```

Example : DIVISION

```
package main
import ("fmt")
func main() {

  x:= 10
  y:= 2

  fmt.Println(x/y)
}
```

Example : MODULO

```
package main
import ("fmt")
func main() {

  x:= 8
  y:= 3

  fmt.Println(x%y)
}
```

Example : INCREMENT

```
package main
import ("fmt")
func main() {

  x:= 10

  x++

  fmt.Println(x)
}
```

Example : DECREMENT

```
package main
import ("fmt")
func main() {

  x:= 10

  x--

  fmt.Println(x)
}
```

Assignment Operators

Assignment operators are used to assign values to variables.

In the example below, we use the **assignment** operator (=) to assign the value **10** to a variable called **x**:

Example

```
package main
import ("fmt")

func main() {
  var x = 10
  fmt.Println(x)
}
```

The **addition assignment** operator (+=) adds a value to a variable:

Example

```
package main
import ("fmt")

func main() {
  var x = 10
  x +=5
  fmt.Println(x)
}
```

A list of all assignment operators:

Operator	Example	Same As	Try it
=	x = 5	x = 5	Try it »
+=	x += 3	x = x + 3	Try it »
-=	x -= 3	x = x - 3	Try it »
*=	x *= 3	x = x * 3	Try it »
/=	x /= 3	x = x / 3	Try it »
%=	x %= 3	x = x % 3	Try it »
&=	x &= 3	x = x & 3	Try it »
|=	x |= 3	x = x | 3	Try it »
^=	x ^= 3	x = x ^ 3	Try it »
>>=	x >>= 3	x = x >> 3	Try it »
<<=	x <<= 3	x = x << 3	Try it »

Comparison Operators

Comparison operators are used to compare two values.
Note: The return value of a comparison is either true (1) or false (0).
In the following example, we use the **greater than** operator (>) to find out if 5 is greater than 3:

Example

```
package main
import ("fmt")

func main() {
  var x = 5
  var y = 3
  fmt.Println(x>y) // returns 1 (true) because 5 is greater than 3
}
```

A list of all comparison operators:

Operator	Name	Example	Try it
==	Equal to	x == y	Try it »
!=	Not equal	x != y	Try it »
>	Greater than	x > y	Try it »
<	Less than	x < y	Try it »
>=	Greater than or equal to	x >= y	Try it »
<=	Less than or equal to	x <= y	Try it »

Logical Operators

- Logical operators are used to determine the logic between variables or values:

Operator	Name	Description	Example	Try it
&&	Logical and	Returns true if both statements are true	x < 5 && x < 10	Try it »
||	Logical or	Returns true if one of the statements is true	x < 5 || x < 4	Try it »
!	Logical not	Reverse the result, returns false if the result is true	!(x < 5 && x < 10)	Try it »

Bitwise Operators

- Bitwise operators are used on (binary) numbers:

Operator	Name	Description	Example	Try it
&	AND	Sets each bit to 1 if both bits are 1	x & y	Try it »
|	OR	Sets each bit to 1 if one of two bits is 1	x | y	Try it »
^	XOR	Sets each bit to 1 if only one of two bits is 1	x ^ b	Try it »
<<	Zero fill left shift	Shift left by pushing zeros in from the right	x << 2	Try it »
>>	Signed right shift	Shift right by pushing copies of the leftmost bit in from the left, and let the rightmost bits fall off	x >> 2	Try it »

```
// Golang program to illustrate  the use of operators
package main
import "fmt"
func main() {
    p := 23
    q := 60
// Arithmetic Operator - Addition
    result1 := p + q
    fmt.Printf("Result of p + q = %d\n", result1)
// Relational Operators - '=='(Equal To)
    result2 := p == q
    fmt.Println(result2)
// Relational Operators - '!='(Not Equal To)
    result3 := p != q
    fmt.Println(result3)
```

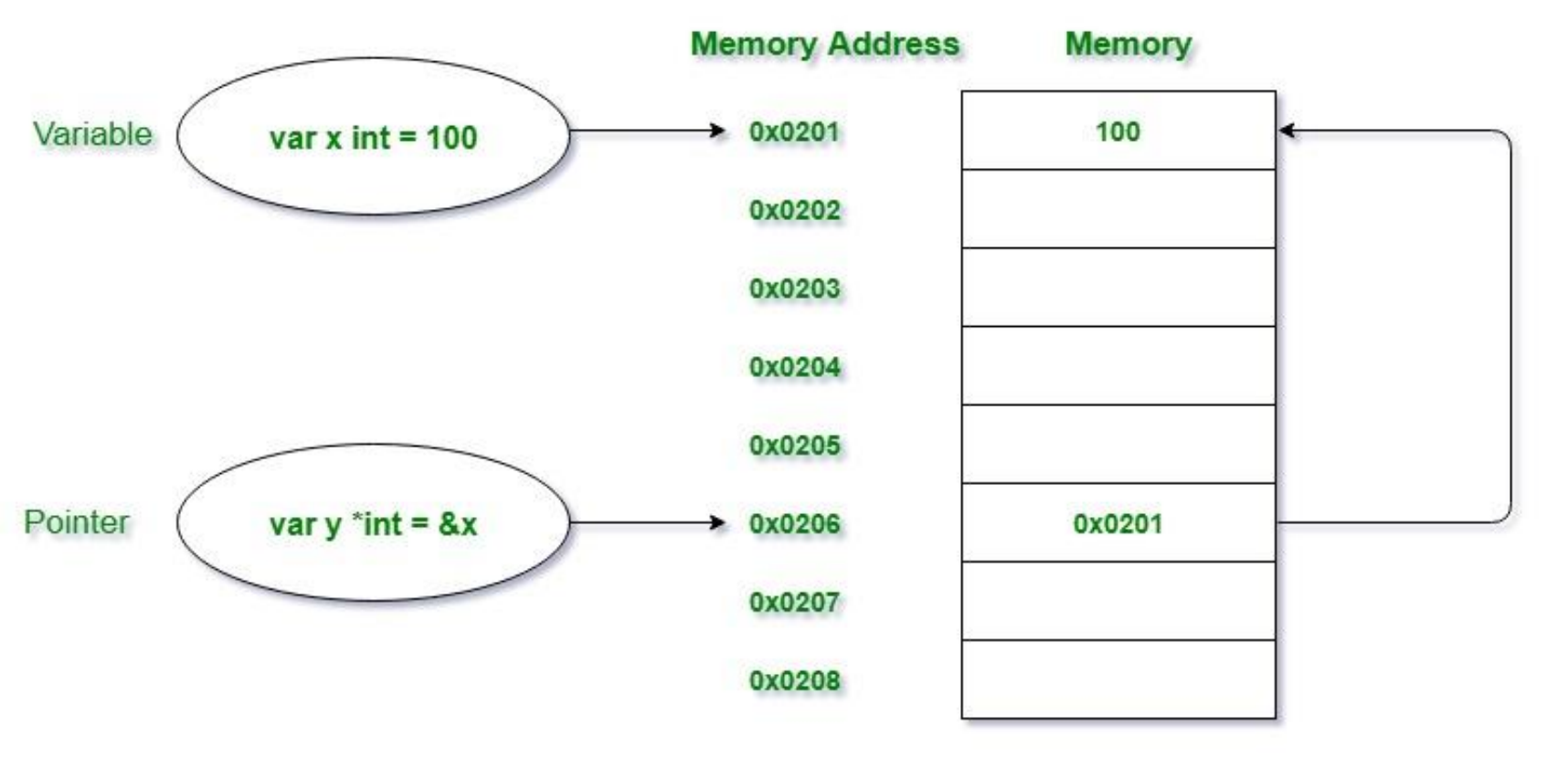
```
// Logical Operators
        if p != q && p <= q {
                fmt.Println("True")
        }
        if p != q || p <= q {
                fmt.Println("True")
        }
        if !(p == q) {
                fmt.Println("True")
        }
// Bitwise Operators - & (bitwise AND)
        result4 := p & q
        fmt.Printf("Result of p & q = %d\n", result4)
// Assignment Operators - "="(Simple Assignment)
        p = q
        fmt.Println(p)
}
```

Output:

```
Result of p + q =
83 false
true
True
True
True
Result of p & q =
20
60
```

Go Pointer

- A pointer is a special kind of variable that is not only used to store the memory addresses of other variables but also points where the memory is located and provides ways to find out the value stored at that memory location.
- It is generally termed as a Special kind of Variable because it is almost declared as a variable but with *(dereferencing operator).



- Before we start there are two important operators which we will use in pointers i.e.
- *** Operator** also termed as the dereferencing operator used to declare pointer variable and access the value stored in the address.
- **& operator** termed as address operator used to returns the address of a variable or to access the address of a variable to a pointer.

// Golang program to demonstrate the declaration and initialization of pointers

```
package main
import "fmt"
func main() {
    // taking a normal variable
    var x int = 5748
    // declaration of pointer
    var p *int
    // initialization of pointer
    p = &x
    // displaying the result
    fmt.Println("Value stored in x = ", x)
    fmt.Println("Address of x = ", &x)
    fmt.Println("Address of x = ", p)
    fmt.Println("Value stored in variable p = ",*p)
}
```

Output:

```
Value stored in x = 5748
Address of x = 0x414020
Address of x = 0x414020
Value stored in variable p = 5748
```

1. The default value or zero-value of a pointer is always **nil**. Or you can say that an uninitialized pointer will always have a nil value.

Example:

```
// Golang program to demonstrate the nil value of the pointer


package main
import "fmt"
func main() {
        var s *int              // taking a pointer
        fmt.Println("s = ", s)      // displaying the result
}
```

O/P:

```
s = <nil>
```

2. Declaration and initialization of the pointers can be done into a single line.

Example:

```
var s *int = &a
```

3. If you are specifying the data type along with the pointer declaration then the pointer will be able to handle the memory address of that specified data type variable.

For example, if you taking a pointer of string type then the address of the variable that you will give to a pointer will be only of string data type variable, not any other type.

4. To overcome the above mention problem you can use the Type Inference concept of the var keyword.

There is no need to specify the data type during the declaration.

The type of a pointer variable can also be determined by the compiler like a normal variable.

Here we will not use the * operator. It will internally determine by the compiler as we are initializing the variable with the address of another variable.

Example:

```
// Golang program to demonstrate  the use of type inference in Pointer variables

package main
import "fmt"
func main() {
  var y = 458        // using var keyword we are not defining any type with variable
  var p = &y  // taking a pointer variable using var keyword without specifying the type

fmt.Println("Value stored in y = ", y)
fmt.Println("Address of y = ", &y)
fmt.Println("Value stored in pointer variable p = ", p)
}
```

Output:

```
Value stored in y = 458
Address of y = 0x414020
Address in variable p = 0x414020
```

- **5.** You can also use the *shorthand (:=)* syntax to declare and initialize the pointer variables. The compiler will internally determine the variable is a pointer variable if we are passing the address of the variable using *&(address)* operator to it.

Example:

```
// Golang program to demonstrate  the use of shorthand syntax in Pointer variables
package main
import "fmt"
func main() {
        // using := operator to declare and initialize the variable
        y := 458
        // taking a pointer variable using := by assigning it with the address of variable y
        p := &y
        fmt.Println("Value stored in y = ", y)
        fmt.Println("Address of y = ", &y)
        fmt.Println("Value stored in pointer variable p = ", p)
}
```

Output:

```
Value stored in y = 458
Address of y = 0x414020
Value stored in pointer variable p = 0x414020
```

Dereferencing the Pointer

As we know that * operator is also termed as the dereferencing operator. It is not only used to declare the pointer variable but also used to access the value stored in the variable which the pointer points to which is generally termed as **indirecting or dereferencing**. * *operator is also termed as the value at the address of*.

Let’s take an example to get a better understandability of this concept:

Example:

```
// Golang program to illustrate the concept of dereferencing a pointer
package main
iport "fmt"
func main() {
    // using var keyword we are not defining any type with variable
    var y = 458
    // taking a pointer variable using var keyword without specifying the type
    var p = &y
    fmt.Println("Value stored in y = ", y)
    fmt.Println("Address of y = ", &y)
    fmt.Println("Value stored in pointer variable p = ", p)
    // this is dereferencing a pointer using * operator before a pointer  variable to access the value stored at the variable at which it is pointing
    fmt.Println("Value stored in y(*p) = ", *p)
}
```

Go Pointers

- Pointers in Go programming language or Golang is a variable that is used to store the memory address of another variable.
- Pointers in Golang is also termed as the special variables. The variables are used to store some data at a particular memory address in the system.
- The memory address is always found in hexadecimal format(starting with 0x like 0xFFAAF etc.).

What is the need for the pointers?

- To understand this need, first, we have to understand the concept of variables.
- Variables are the names given to a memory location where the actual data is stored.
- To access the stored data we need the address of that particular memory location.
- To remember all the memory addresses(Hexadecimal Format) manually is an overhead that's why we use variables to store data and variables can be accessed just by using their name.
- Golang also allows saving a hexadecimal number into a variable using the literal expression i.e. number starting from **0x** is a hexadecimal number.

- **Example:**
- In the below program, we are storing the hexadecimal number into a variable.
- But you can see that the type of values is *int* and saved as the decimal or you can say the decimal value of type *int* is storing.
- But the main point to explain this example is that we are storing a hexadecimal value(consider it a memory address) but it is not a pointer as it is not pointing to any other memory location of another variable.
- It is just a user-defined variable. So this arises the need for pointers.

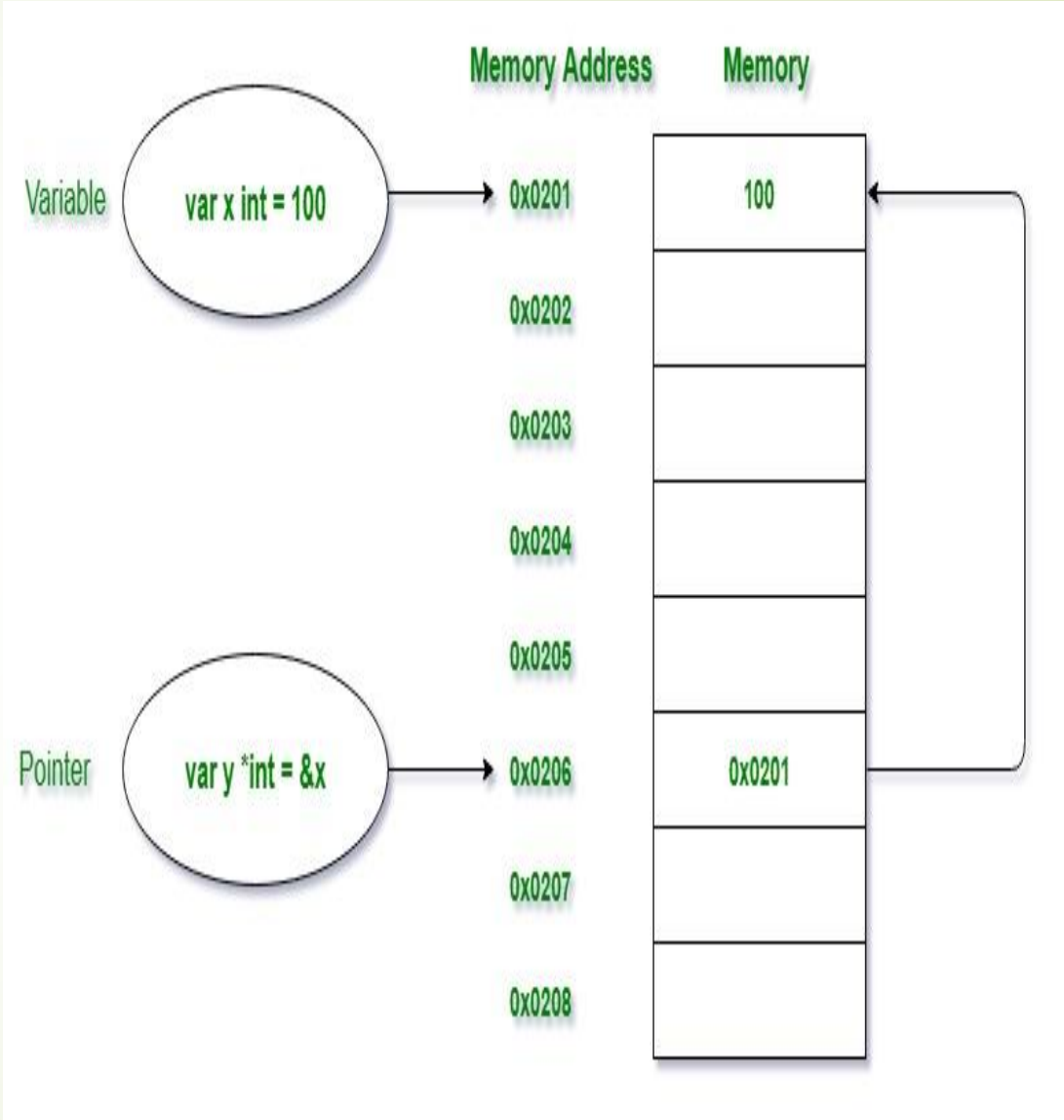
```
// Golang program to demonstrate the variables
// storing the hexadecimal values
package main
import "fmt"
func main() {
    // storing the hexadecimal values in variables
    x := 0xFF
    y := 0x9C
    // Displaying the values
    fmt.Printf("Type of variable x is %T\n", x)
    fmt.Printf("Value of x in hexadecimal is %X\n", x)
    fmt.Printf("Value of x in decimal is %v\n", x)
    fmt.Printf("Type of variable y is %T\n", y)
    fmt.Printf("Value of y in hexadecimal is %X\n",
y)
    fmt.Printf("Value of y in decimal is %v\n", y)}
```

Output:

```
Type of variable x is int
Value of x in hexadecimal is FF
Value of x in decimal is 255
Type of variable y is int
Value of y in hexadecimal is 9C
Value of y in decimal is 156
```

- A pointer is a special kind of variable that is not only used to store the memory addresses of other variables but also points where the memory is located and provides ways to find out the value stored at that memory location.

- It is generally termed as a Special kind of Variable because it is almost declared as a variable but with *(dereferencing operator).



- Before we start there are two important operators which we will use in pointers i.e.
- *** Operator** also termed as the dereferencing operator used to declare pointer variable and access the value stored in the address.
- **& operator** termed as address operator used to returns the address of a variable or to access the address of a variable to a pointer.
- ***Declaring a pointer***:

```
var pointer_name *Data_Type
```

Initialization of Pointer: To do this you need to initialize a pointer with the memory address of another variable using the address operator as shown in the below example:

```
// normal variable declaration
var a = 45

// Initialization of pointer s with memory address of //variable a
var s *int = &a
```

```
// Golang program to demonstrate the declaration and initialization of pointers
package main
import "fmt"
func main() {
	// taking a normal variable
	var x int = 5748
	// declaration of pointer
	var p *int
	// initialization of pointer
	p = &x
		// displaying the result
	fmt.Println("Value stored in x = ", x)
	fmt.Println("Address of x = ", &x)
	fmt.Println("Value stored in variable p = ", p)
}
```

Output:

```
Value stored in x = 5748 Address of x =
0x414020 Value stored in variable p =
0x414020
```

Important Points

1. The default value or zero-value of a pointer is always **nil**. Or you can say that an uninitialized pointer will always have a nil value.

Example:

```
// Golang program to demonstrate the nil value of the pointer
package main
import "fmt"
func main() {

    // taking a pointer
    var s *int

    // displaying the result
    fmt.Println("s = ", s)
}
```

Output:

```
s = <nil>
```

2. Declaration and initialization of the pointers can be done into a single line.

Example:

```
var s *int = &a
```

3. If you are specifying the data type along with the pointer declaration then the pointer will be able to handle the memory address of that specified data type variable.

For example, if you taking a pointer of string type then the address of the variable that you will give to a pointer will be only of string data type variable, not any other type.

4. To overcome the above mention problem you can use the Type Inference concept of the var keyword.

There is no need to specify the data type during the declaration. The type of a pointer variable can also be determined by the compiler like a normal variable.

Here we will not use the * operator. It will internally determine by the compiler as we are initializing the variable with the address of another variable.

Example

```
// Golang program to demonstrate  the use of type inference in Pointer variables
package main
import "fmt"
func main() {
    // using var keyword we are not defining any type with variable
    var y = 458
    // taking a pointer variable using var keyword without specifying the type
    var p = &y
    fmt.Println("Value stored in y = ", y)
    fmt.Println("Address of y = ", &y)
    fmt.Println("Value stored in pointer variable p = ", p)
}
```

Output:

```
Value stored in y = 458 Address of y =
0x414020 Value stored in pointer variable p
= 0x414020
```

5. You can also use the *shorthand (:=)* syntax to declare and initialize the pointer variables. The compiler will internally determine the variable is a pointer variable if we are passing the address of the variable using *&(address)* operator to it.

```
// Golang program to demonstrate the use of shorthand syntax in Pointer variables
package main
import "fmt"
func main() {
    // using := operator to declare and initialize the variable
    y := 458

    // taking a pointer variable using := by assigning it with the address of variable y
    p := &y
    fmt.Println("Value stored in y = ", y)
    fmt.Println("Address of y = ", &y)
    fmt.Println("Value stored in pointer variable p = ", p)
}
```

Output:

```
Value stored in y = 458
Address of y = 0x414020
Value stored in pointer variable p = 0x414020
```

Dereferencing the Pointer

As we know that * operator is also termed as the dereferencing operator. It is not only used to declare the pointer variable but also used to access the value stored in the variable which the pointer points to which is generally termed as **indirecting or dereferencing**. ** operator is also termed as the value at the address of*.

Let’s take an example to get a better understandability of this concept:

Example:

```
// Golang program to illustrate the  concept of dereferencing a pointer
package main
import "fmt"
func main() {
	// using var keyword we are not defining any type with variable
	var y = 458

	// taking a pointer variable using var keyword without specifying the type
	var p = &y

	fmt.Println("Value stored in y = ", y)
	fmt.Println("Address of y = ", &y)
	fmt.Println("Value stored in pointer variable p = ", p)

	// this is dereferencing a pointer using * operator before a pointer variable to access the value
stored
	// at the variable at which it is pointing
	fmt.Println("Value stored in y(*p) = ", *p)

}
```

Output:

```
Value stored in y = 458 Address of y =
0x414020 Value stored in pointer variable p
= 0x414020 Value stored in y(*p) = 458
```

You can also change the value of the pointer or at the memory location instead of assigning a new value to the variable.

Example:

```
// Golang program to illustrate the above mentioned concept
package main
import "fmt"
func main() {
	// using var keyword we are not defining any type with variable
	var y = 458
	// taking a pointer variable using var keyword without specifying the type
	var p = &y
	fmt.Println("Value stored in y before changing = ", y)
	fmt.Println("Address of y = ", &y)
	fmt.Println("Value stored in pointer variable p = ", p)
// this is dereferencing a pointer using * operator before a pointer variable to access the value stored at the variable at which it is pointing
	fmt.Println("Value stored in y(*p) Before Changing = ", *p)
	// changing the value of y by assigning the new value to the pointer
	*p = 500
	fmt.Println("Value stored in y(*p) after Changing = ",y)
}
```

Output:

```
Value stored in y before changing = 458
Address of y = 0x414020
Value stored in pointer variable p =
0x414020
Value stored in y(*p)
Before Changing = 458
Value stored in y(*p) after Changing = 500
```

Pointers to a Function in Go

Create a pointer and simply pass it to the Function

In the below program we are taking a function *ptf* which have integer type pointer parameter which instructs the function to accept only the pointer type argument.

Basically, this function changed the value of the variable *x*.

At starting *x* contains the value 100.

But after the function call, value changed to 748 as shown in the output.

```
// Go program to create a pointer and passing it to the function
package main
import "fmt"
// taking a function with integer type pointer as an parameter
func ptf(a *int) {
        // dereferencing
    *a = 748
}
```

```
// Main function
func main() {
    // taking a normal variable
    var x = 100
    fmt.Printf("The value of x before function call is: %d\n", x)
    // taking a pointer variable and assigning the address of x to it
    var pa *int = &x
    // calling the function by passing pointer to function
    ptf(pa)
    fmt.Printf("The value of x after function call is: %d\n", x)
}
```

Output:

```
The value of x before function call is: 100
The value of x after function call is: 748
```

Passing an address of the variable to the function call

- Considering the below program, we are not creating a pointer to store the address of the variable *x* i.e. like *pa* in the above program.
- We are directly passing the address of **x** to the function call which works like the above-discussed method.

```
// Go program to create a pointer and passing the address of the variable to the function
package main
import "fmt"
// taking a function with integer type pointer as an parameter
func ptf(a *int) {
        // dereferencing
        *a = 748
}
// Main function

func main() {
        // taking a normal variable
        var x = 100
        fmt.Printf("The value of x before function call is: %d\n", x)
        // calling the function by passing the address of the variable x
        ptf(&x)
        fmt.Printf("The value of x after function call is: %d\n", x)
}
```

Output:

```
The value of x before function call is: 100
The value of x after function call is: 748
```

Note: You can also use the short declaration operator(:=) to declare the variables and pointers in above programs.

Strings in Golang

- In Go language, strings are different from other languages like Java, C++, Python, etc. it is a sequence of variable-width characters where each and every character is represented by one or more bytes using UTF-8 Encoding.
- Or in other words, strings are the immutable chain of arbitrary bytes(including bytes with zero value) or ***string is a read-only slice of bytes*** and the bytes of the strings can be represented in the Unicode text using UTF-8 encoding.
- Due to UTF-8 encoding Golang string can contain a text which is the mixture of any language present in the world, without any confusion and limitation of the page. Generally, strings are enclosed in *double-quotes""*, as shown in the below example:

Example:

```
// Go program to illustrate how to create strings
package main
import "fmt"
func main() {
    // Creating and initializing a variable with a string Using shorthand declaration
    My_value_1 := "Welcome to GeeksforGeeks"

    // Using var keyword
    var My_value_2 string
    My_value_2 = "GeeksforGeeks"

    // Displaying strings
    fmt.Println("String 1: ", My_value_1)
    fmt.Println("String 2: ", My_value_2)
}
```

Output:

```
String 1: Welcome to GeeksforGeeks
String 2: GeeksforGeeks
```

Note: String can be empty, but they are not nil.

String Literals

- In Go language, string literals are created in two different ways:
- **Using double quotes(“”):** Here, the string literals are created using double-quotes(“”).
- This type of string support escape character as shown in the below table, but does not span multiple lines.
- This type of string literals is widely used in Golang programs.

Escape character	Description
\\	Backslash(\)
\000	Unicode character with the given 3-digit 8-bit octal code point
\'	Single quote (‘). It is only allowed inside character literals
\"	Double quote (“). It is only allowed inside interpreted string literals
\a	ASCII bell (BEL)
\b	ASCII backspace (BS)
\f	ASCII formfeed (FF)
\n	ASCII linefeed (LF
\r	ASCII carriage return (CR)
\t	ASCII tab (TAB)
\uhhhh	Unicode character with the given 4-digit 16-bit hex code point.
	Unicode character with the given 8-digit 32-bit hex code point.
\v	ASCII vertical tab (VT)
\xhh	Unicode character with the given 2-digit 8-bit hex code point.

- **Using backticks(“):** Here, the string literals are created using backticks(“) and also known as **raw literals**.
- Raw literals do not support escape characters, can span multiple lines, and may contain any character except backtick.
- It is, generally, used for writing multiple line message, in the regular expressions, and in HTML.

Example:

```
// Go program to illustrate string literals
package main
import "fmt"
func main() {
    // Creating and initializing a variable with a string literal  Using double-quote
    My_value_1 := "Welcome to GeeksforGeeks"
    // Adding escape character
    My_value_2 := "Welcome!\nGeeksforGeeks"
    // Using backticks
    My_value_3 := `Hello!GeeksforGeeks`
    // Adding escape character in raw literals
    My_value_4 := `Hello!\nGeeksforGeeks`
    // Displaying strings
    fmt.Println("String 1: ", My_value_1)
    fmt.Println("String 2: ", My_value_2)
    fmt.Println("String 3: ", My_value_3)
    fmt.Println("String 4: ", My_value_4)
}
```

Output:

```
String 1: Welcome to GeeksforGeeks
String 2: Welcome! GeeksforGeeks
String 3: Hello!GeeksforGeeks
String 4: Hello!\nGeeksforGeeks
```

Important Points About String

Strings are immutable: In Go language, strings are immutable once a string is created the value of the string cannot be changed. Or in oth// Go program to illustrate string are immutable

```
package main
import "fmt"
// Main function
func main() {
	// Creating and initializing a string using shorthand declaration
	mystr := "Welcome to GeeksforGeeks"
	fmt.Println("String:", mystr)
	/* if you trying to changethe v alue of the string then the compiler will throw an error, i.e, cannot assign to mystr[1]
	mystr[1]= 'G'
	fmt.Println("String:", mystr)
	*/
}
```

er words, strings are read-only. If you try to change, then the compiler will throw an error.

Example:

Control Flow
Decision Making Statements

- Decision Making in programming is similar to decision making in real life.
- A piece of code is executed when the given condition is fulfilled.
- Sometimes these are also termed as the Control flow statements.
- A programming language uses control statements to control the flow of execution of the program based on certain conditions.
- These are used to cause the flow of execution to advance and branch based on changes to the state of a program.

Go If

- The if statement in Go is used to test the condition. If it evaluates to true, the body of the statement is executed. If it evaluates to false, if block is skipped.
- Syntax :

```
if(boolean_expression) {
   /* statement(s) got executed only if the expression results in true */
      }
```

Example : if

```
package main
import "fmt"
func main() {
   /* local variable definition */
   var a int = 10
   /* check the boolean condition using if statement */
   if( a % 2==0 ) {      /* if condition is true then print the following
*/ fmt.Printf("a is even number" )
   }
}
```

o/p

a is even number

Go if-else

The if-else is used to test the condition. If condition is true, if block is executed otherwise else block is executed.

Syntax :

```
if(boolean_expression) {
   /* statement(s) got executed only if the expression results in true */
} else {
   /* statement(s) got executed only if the expression results in false */
}
```

Go if-else example

```
package main
import "fmt"
func main() {
    var a int = 10;
    if ( a%2 == 0 )
  {
          fmt.Printf("a is even\n");
   }
 else
  {
         fmt.Printf("a is odd\n");
   }
   fmt.Printf("value of a is : %d\n", a);
}
```

Output:

a is even value of a is : 10

Go If-else example: with input from user

```
func main() {
   fmt.Print("Enter number: ")
   var input int
  fmt.Scanln(&input)
   fmt.Print(input)
     if( input % 2==0 )
   {
       fmt.Printf(" is even\n" );
   }
else
   {
          fmt.Printf(" is odd\n" );
   }
}
```

Output:

Enter number: 10 10 is even

Go If else-if chain

The Go if else-if chain is used to execute one statement from multiple conditions.

We can have N numbers of if-else statement. It has no limit.

The curly braces{ } are mandatory in if-else statement even if you have one statement in it.

The else-if and else keyword must be on the same line after the closing curly brace }.

Go If else-if chain Example

```
package main
import "fmt"
func main() {
   fmt.Print("Enter text: ")
   var input int
   fmt.Scanln(&input)
   if (input < 0 || input > 100) {
      fmt.Print("Please enter valid no")
   } else if (input >= 0 && input < 50  ) {
      fmt.Print(" Fail")
   } else if (input >= 50 && input < 60) {
      fmt.Print(" D Grade")
   } else if (input >= 60 && input < 70  ) {
      fmt.Print(" C Grade")
   } else if (input >= 70 && input < 80) {
      fmt.Print(" B Grade")
   } else if (input >= 80 && input < 90  ) {
      fmt.Print(" A Grade")
   } else if (input >= 90 && input <= 100) {
      fmt.Print(" A+ Grade")
   }
}
```

Output:
Enter text: 84 A Grade

Go Nested if-else

- We can also nest the if-else statement to execute one statement from multiple conditions.

Syntax

```
if( boolean_expression 1) {
/* statement(s) got executed only if the expression 1 results in tru
e */
   if(boolean_expression 2) {
/* statement(s) got executed only if the expression 2 results in tru
e */
   }
}
```

nested if-else example

```
package main
import "fmt"
func main() {
   var x int = 10
   var y int = 20
   if( x >=10 )
   {
            if( y >= 10 )
      {
               fmt.Printf("Inside nested If Statement \n" );
      }
   }
   fmt.Printf("Value of x is : %d\n", x );
   fmt.Printf("Value of y is : %d\n", y );
}
```

Output:

Inside nested If Statement Value of x is : 10 Value of y is : 20

Go switch

The Go **switch statement** executes one statement from multiple conditions. It is similar to if-else-if chain statement.

Syntax:

```
switch  var1 {
case val1:
.....
case val2
.....
default:
.....
}
```

- The switch statement in Go is more flexible. In the above syntax, var1 is a variable which can be of any type, and val1, val2, ... are possible values of var1.
- In switch statement, more than one values can be tested in a case, the values are presented in a comma separated list

  like: case val1, val2, val3:
- If any case is matched, the corresponding case statement is executed. Here, the break keyword is implicit. So automatic fall-through is not the default behavior in Go switch statement.
- For fall-through in Go switch statement, use the keyword "fallthrough" at the end of the branch.

Go Switch Example:

```
package main
import "fmt"
func main() {
   fmt.Print("Enter Number: ")
  var input int
  fmt.Scanln(&input)
   switch (input) {
  case 10:
      fmt.Print("the value is 10")
    case 20:
      fmt.Print("the value is 20")
    case 30:
      fmt.Print("the value is 30")
    case 40:
      fmt.Print("the value is 40")
    default:
      fmt.Print(" It is not 10,20,30,40 ")
   }
}
```

Output:
Enter Number: 20 the value is 20

or

Output:
Enter Number: 35 It is not 10,20,30,40

Go switch fallthrough example

```
import "fmt"
func main() {
   k := 30
   switch k {
   case 10:
      fmt.Println("was <= 10"); fallthrough;
   case 20:
      fmt.Println("was <= 20"); fallthrough;
   case 30:
      fmt.Println("was <= 30"); fallthrough;
   case 40:
      fmt.Println("was <= 40"); fallthrough;
   case 50:
      fmt.Println("was <= 50"); fallthrough;
   case 60:
      fmt.Println("was <= 60"); fallthrough;
   default:
      fmt.Println("default case")
   }
}
```

Output:

```
was <= 30
was <= 40
was <= 50
was <= 60
default case
```

Go For Loop

- The Go for statement is used for repeating a set of statements number of times. It is the only loop in go language.
- There are two variants of for loop in Go: Counter-controlled iteration and Condition-controlled iteration.
- When the execution of the loop is over, the objects created inside the loop gets destroyed.

Go For Loop counter-controlled iteration example:

```
package main

import "fmt"

func main() {

   for a := 0; a < 11; a++ {

      fmt.Print(a,"\n")

   }

}
```

As you can see in the above example, loop begins with the initialization stage, variable for i (i:= 0); This is done only once.

It is followed by a conditional check i (i < 10). Conditional check is performed in every iteration.

The for loop stops when the condition becomes false.

Output:

```
0
1
2
3
4
5
6
7
8
9
10
```

```go
package main
import "fmt"
func main() {
   for a := 0; a < 3; a++ {
      for b := 3;b > 0; b-- {
         fmt.Print(a," ",b,"\n")
      }
   }
}
```

Output:

```
0 3
0 2
0 1
1 3
1 2
1 1
2 3
2 2
2 1
```

Go Infinitive For Loop

In infinite for loop, the conditional statement is absent like:

```
for i:=0; ; i++
```

or

```
for { }
```

Example:

```
package main
import "fmt"
func main() {
  for true  {
    fmt.Printf("This loop will run forever.\n");
  }
}
```

Output:

```
This loop will run forever.
This loop will run forever.
This loop will run forever.
This loop will run forever.
```

Go For - Condition-controlled iteration

The for loop which has no header is used for condition-controlled iteration. It is similar to while-loop in other languages.

Syntax:

```
for condition { }
```

For Loop Example in while fashion:

```
package main
import "fmt"
func main() {
   sum := 1
   for sum < 100 {
      sum += sum
      fmt.Println(sum)
   }
}
```

Output:

2
4
8
16
32
64
128

Range Keyword in Golang

- In Golang **Range** keyword is used in different kinds of data structures in order to iterates over elements.
- The **range** keyword is mainly used in for loops in order to iterate over all the elements of a **map**, **slice**, **channel**, or an **array**.
- When it iterates over the elements of an array and slices then it returns the index of the element in an integer form.
- And when it iterates over the elements of a map then it returns the key of the subsequent key-value pair.
- Moreover, *range* can either returns one value or two values.
- *Lets see what range returns while iterating over different kind of collections in Golang.*

```
// Golang Program to illustrate the usage
// of range keyword over items of an
// array in Golang
package main
import "fmt"
// main function
func main() {
	// Array of odd numbers

	odd := [7]int{1, 3, 5, 7, 9, 11, 13}
	// using range keyword with for loop to iterate over the array elements

	for i, item := range odd {
		// Prints index and the elements
		fmt.Printf("odd[%d] = %d \n", i, item)
	}
}
```

Output:

```
odd[0] = 1
odd[1] = 3
odd[2] = 5
odd[3] = 7
odd[4] = 9
odd[5] = 11
odd[6] = 13
```

Here, all the elements are printed with their respective index.

```
// Golang Program to illustrate the usage of range keyword
over string in Golang
package main
import "fmt"
// Constructing main function
func main() {
    // taking a string
    var string = "GeeksforGeeks"
    // using range keyword with for loop to iterate over the
string

    for i, item := range string {
        // Prints index of all the characters in the string
        fmt.Printf("string[%d] = %d \n", i, item)
    }
}
```

Output:

```
string[0] = 71
string[1] = 101
string[2] = 101
string[3] = 107
string[4] = 115
string[5] = 102
string[6] = 111
string[7] = 114
string[8] = 71
string[9] = 101
string[10] = 101
string[11] = 107
string[12] = 115
```

Here, the items printed is the *rune*, that is *int32* ASCII value of the stated characters that forms string.

Go for range construct

The for range construct is useful in many context. It can be used to traverse every item in a collection.

It is similar to foreach in other languages. But, we still have the index at each iteration in for range construct.

Syntax:

```
for ix, val := range coll { }
```

Go For Range Example

```
import "fmt"
func main() {
   nums := []int{2, 3, 4}
   sum := 0
   for _, value := range nums {// "_ " is to ignore the index
      sum += value
   }
```

```
fmt.Println("sum:", sum)
  for i, num := range nums {
     if num == 3 {
        fmt.Println("index:", I)
     }
  }
```

```
kvs := map[string]string{"1":"mango","2":"apple","3":"banana"}
    for k, v := range kvs {
        fmt.Printf("%s -> %s\n", k, v)
    }
    for k := range kvs {
        fmt.Println("key:", k)
    }
    for i, c := range "Hi" {
        fmt.Println(i, c)
    }
}
```

- Output:

```
sum: 60
1 -> mango
2 -> apple
3 -> banana
key: 1
key: 2
key: 3
0 72
1 105
```

Go Break Statement

A break statement is used to break out of the innermost structure in which it occurs. It can be used for-loop (counter, condition,etc.), and also in a switch. Execution is continued after the ending } of th structure.

Syntax:-

```
break;
```

Go Break Statement Example:

```
package main
import "fmt"
func main() {
   var  a int = 1
   for a < 10{
      fmt.Print("Value of a is ",a,"\n")
      a++;
      if a > 5{
         /* terminate the loop using break statement */
         break;
      }
   } }
```

Output:

Value of a is 1

Value of a is 2

Value of a is 3

Value of a is 4

Value of a is 5

Break statement can also be applied in the inner loop, and the control flow break out to the outer loop.

Go Break Statement with Inner Loop:

```
package main
import "fmt"
func main() {
   var a int
   var b int
   for a = 1; a <= 3; a++ {
      for b = 1; b <= 3; b++ {
         if (a == 2 && b == 2) {
            break;
         }
         fmt.Print(a, " ", b, "\n")
    }    } }
```

Output:

```
1 1
1 2
1 3
2 1
3 1
3 2
3 3
```

Go Continue Statement

The continue is used to skip the remaining part of the loop, and then continues with the next iteration of the loop after checking the condition.

- Syntax:-

```
continue;
```

Or we can do like

```
x:

continue:x
```

Go Continue Statement Example:

```
package main
import "fmt"
func main() {
   /* local variable definition */
   var a int = 1
   /* do loop execution */
   for a < 10 {
      if a == 5 {
         /* skip the iteration */
         a = a + 1;
         continue;
      }
      fmt.Printf("value of a: %d\n", a);
      a++;
   }
}
```

Output:

```
value of a: 1
value of a: 2
value of a: 3
value of a: 4
value of a: 6
value of a: 7
value of a: 8
value of a: 9
```

Continue can be also be applied in the inner loop
Go Continue Statement with Inner Loop example:

```
package main
import "fmt"
func main() {
  var a int = 1  /* local variable definition */
   var b int = 1
   /* do loop execution */
   for a = 1; a < 3; a++ {
      for b = 1; b < 3; b++ {
         if a == 2 && b == 2 {
            /* skip the iteration */
            continue;
         }
         fmt.Printf("value of a and b is %d %d\n", a, b);
      }
      fmt.Printf("value of a and b is %d %d\n", a, b);    }  }
```

Output:

value of a and b is 1 1
value of a and b is 1 2
value of a and b is 1 3
value of a and b is 2 1
value of a and b is 2 3